

BAIGIAMOJO DARBO ATASKAITA

Išmani Steam žaidimų paieškos ir rekomendacijų sistema

Semantiniai embeddings • FAISS • SVD • NCF • RAG • SHAP

Kęstas Trybė

2026-04-15

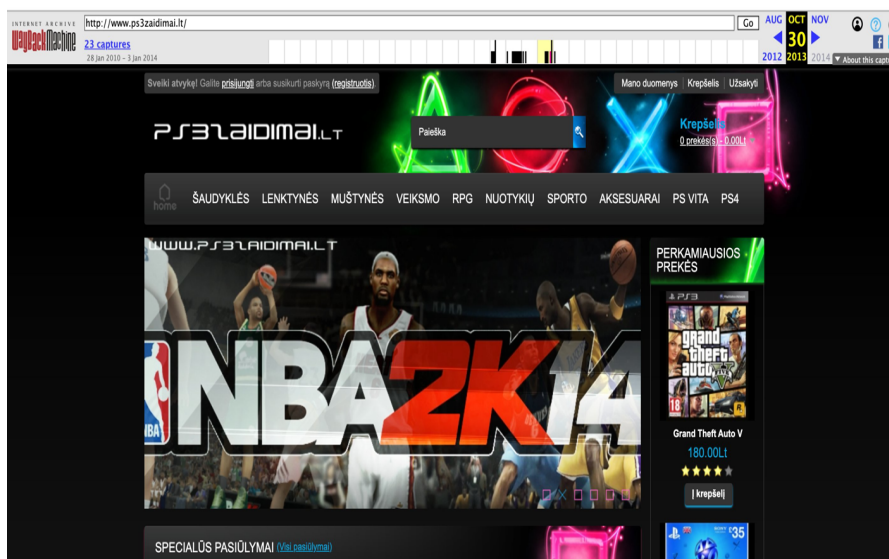
1. Įvadas

Šis baigiamasis darbas pristatomas kaip Code Academy dirbtinio intelekto kurso (AIU6) galutinis projektas. Darbo tikslas – sukurti išmaniąją Steam žaidimų paieškos ir rekomendacijų sistemą, kuri apjungia modernias dirbtinio intelekto technologijas į vieningą, veikiančią programą.

1.1 Asmeninė motyvacija

Šio projekto tema nėra atsitiktinė. 2009–2015 metais valdžiau elektroninę PS3 žaidimų parduotuvę ps3zaidimai.lt – vieną pirmųjų lietuviškų specializuotų žaidimų e-parduotuvių. Parduotuvė veikė iki PS4 kartos atėjimo, kai rinka smarkiai pasikeitė.

Dirbdamas su žaidimų katalogu iš arti supratau pagrindinę problemą: klientai dažnai nežinojo ko ieško – turėjo nuojautą ('niūrus veiksmo žaidimas', 'su draugais prie sofos'), bet klasikiniai filtrai (žanras, kaina) nepadėdavo rasti tiksliai tinkamo žaidimo. Šis projektas yra atsakymas į tą problemą – tik šį kartą su dirbtinio intelekto įrankiais.



Pav. ps3zaidimai.lt elektroninė parduotuvė (2013 m. spalio) – archyvas pasiekiamas per web.archive.org/web/*/ps3zaidimai.lt

1.2 Problematika

Steam platformoje yra daugiau nei 50 000 žaidimų. Vartotojui rasti tinkamą žaidimą pagal aprašymą, nuotaką ar žaidimo stilių yra sudėtinga: paprasta raktažodžių paieška nesupranta prasmės, o algoritminio filtravimo galimybės ribotos. Šia sistema siekiama išspręsti šią problemą pasitelkiant semantinę vektorinę paiešką ir kolaboratyvų filtravimą.

1.3 Tikslai

- Sukurti semantinę paiešką, kuri supranta vartotojo ketinimą, ne tik raktinius žodžius
- Integruoti kolaboratyvojo filtravimo modelius (SVD ir NCF) žaidimų rekomendacijoms
- Paaiškinti rekomendacijas vartotojui naudojant SHAP ir RAG technologijas

- Sukurti veikiančią Flask web aplikaciją su pilna UI
- Palyginti neuroninius ir ne-neuroninius modelius kiekybiniais rodikliais

1.5 Technologijų apžvalga

Technologija	Paskirtis	Etapas
Python 3.11 + Flask	Backend + Web UI	Visas projektas
PostgreSQL + SQLAlchemy 2.0	Duomenų bazė	1 etapas
sentence-transformers (all-mpnet-base-v2)	768d semantiniai embeddings	3 etapas
FAISS IndexFlatL2	Vektorinė paieška	3 etapas
TruncatedSVD (scikit-learn)	Kolaboratyvus filtravimas	4 etapas
NCF (PyTorch)	Neuroninis kolaboratyvus filtravimas	4 etapas
Gemini 2.5 Flash (Google AI)	RAG paaiškinimų generavimas	5 etapas
SHAP + GradientBoosting	Explainable AI	6 etapas
Pandas / NumPy / Seaborn	Duomenų apdorojimas ir EDA	1–2 etapas

2. Duomenys ir duomenų bazė

2.1 Duomenų rinkiniai

Projektui naudoti trys duomenų rinkiniai, visi gauti iš Kaggle platformos:

Failas	Paskirtis	Dydis	Komentaras
games3.csv	Pagrindiniai žaidimų duomenys	~50 872 žaidimų	Kaina, reitingas, platforma
games_papildomas.csv	Papildoma žaidimų info	~39 000 žaidimų	Žanrai, tags, aprašymai, header_image
recommendations3.csv	Vartotojų apžvalgos	~7 mln. įrašų	user_id, app_id, hours, is_recommended

2.2 Duomenų bazės schema

Visi duomenys saugomi PostgreSQL duomenų bazėje. SQLAlchemy 2.0 naudojamas kaip ORM sluoksnis, psycopg2 – masiniam batch įkėlimui (execute_values).

Lentelė	Svarbiausi stulpeliai	Komentaras
games	app_id (PK), title, genres, tags, about_the_game, price_final, positive_ratio, user_reviews, header_image	~50 872 eilučių po deduplication
users	user_id (PK), products, reviews	Filtruota: tik vartotojai su ≥50 apžvalgomis
recommendations	review_id (PK), app_id (FK), user_id (FK), hours, is_recommended	FK validacija prieš įkėlimą
search_cache	query_hash (PK), query_text, result_json	MD5 hash cache – pagreitina paiešką
model_results	model_name, precision_k, recall_k, trained_at	SVD vs NCF metrikų istorija
search_history	session_id, query, results, created_at	Personalizacijai – vartotojo paieškų istorija

2.3 Duomenų valymas ir paruošimas

Keletas svarbių žingsnių duomenų paruošimo metu:

- games_papildomas.csv nuskaitytas su index_col=0 ir reset_index() – išspręsta AppID neatitikimo problema
- app_id konvertuotas su pd.to_numeric(errors='coerce') + filtracija NaN ir inf reikšmių
- LEFT JOIN tarp games3.csv ir games_papildomas.csv per app_id – apjungta genres, tags, header_image info
- Vartotojai filtruoti pagal aktyvumą (≥50 apžvalgų) – sumažintas triukšmas rekomendacijų modeliuose

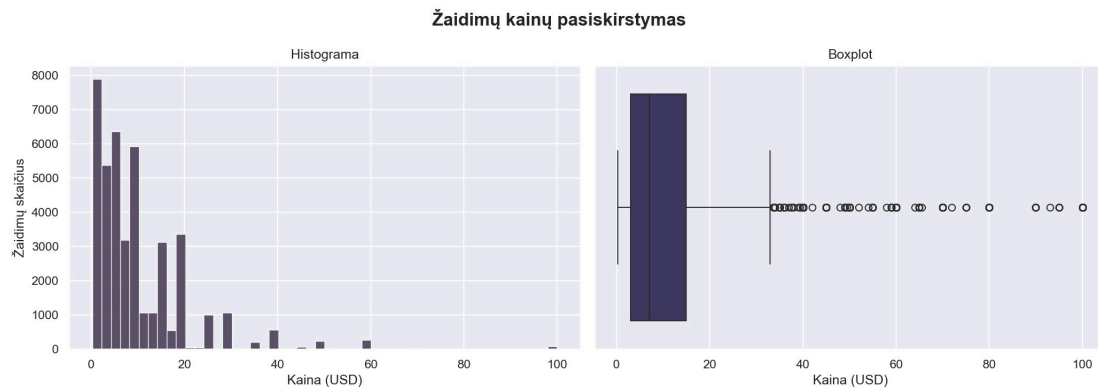
- Žaidimai filtruoti pagal populiarumą (≥ 200 apžvalgų NCF modeliui, ≥ 1000 geriausiai v28)

3. Eksploracinė duomenų analizė (EDA)

EDA atlikta su Pandas, Seaborn ir Matplotlib. Visi grafikai generuojami tiesiogiai iš PostgreSQL duomenų bazės per SQLAlchemy užklausas. Iš viso sugeneruoti 8 grafikai, išsaugoti į `eda_output/` aplanką.

3.1 Kainų pasiskirstymas

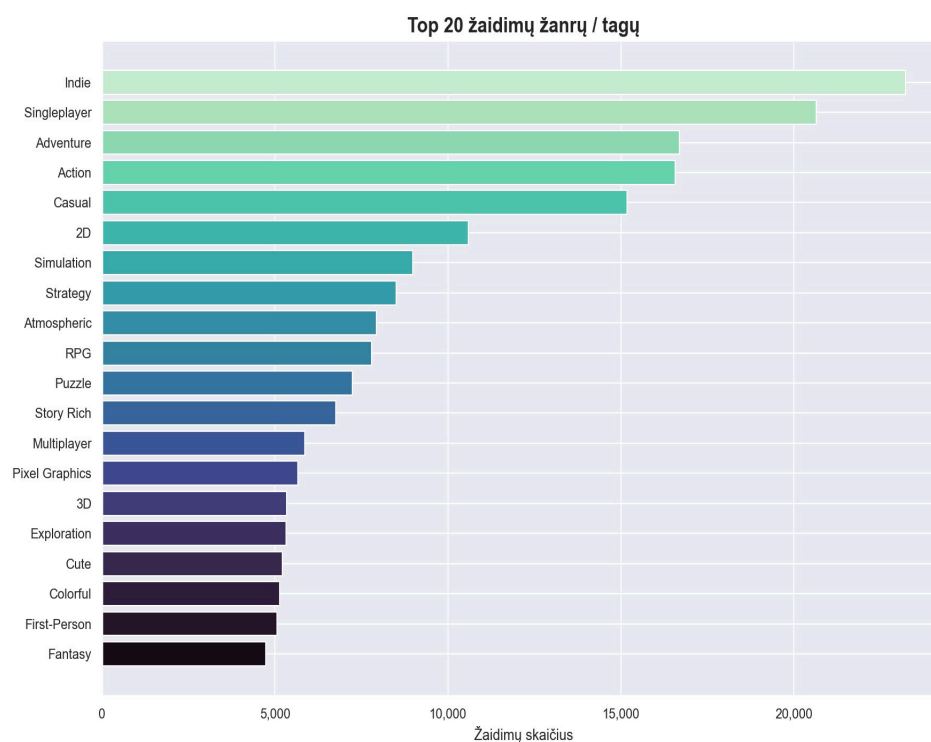
- Histograma: kainos stipriai pasvirusios į dešinę (right-skewed) – dauguma žaidimų kainuoja \$0–\$10
- Boxplot: dauguma žaidimų IQR \$2–\$15, daug outlierių virš \$30
- Nemokamų žaidimų (`price_final = 0`) filtruota iš šio grafiko, bet analizuota atskirai (grafike 07)
- Ekstremalios reikšmės ($> \$60$) – retos, bet egzistuoja rinkoje



Pav. 2. Žaidimų kainų pasiskirstymas – histograma ir boxplot

3.2 Žanrų ir tagų analizė

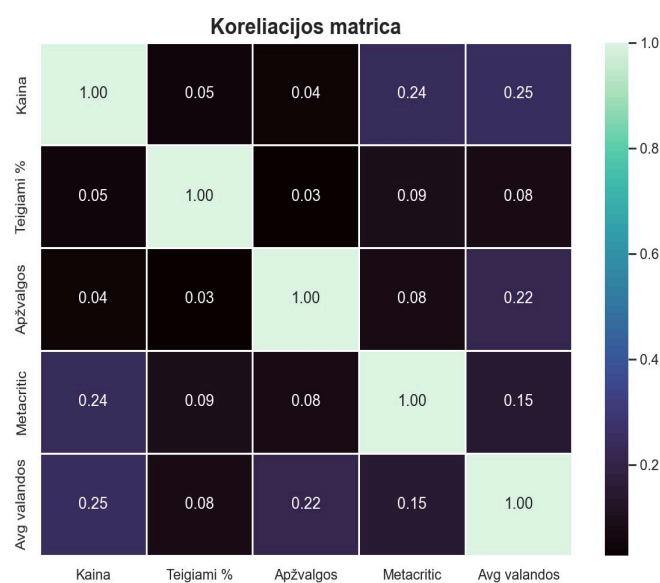
- Analizuotas tags stulpelis (detalesnė kategorija nei genres)
- Top 1: Indie (~23 000 žaidimų) – dominuoja dėl žemų kūrimo barjerų Steam platformoje
- Top 2–4: Singleplayer, Adventure, Action – ~16 000–21 000 žaidimų kiekvienas
- Daugelis žaidimų turi 5–15 tagų – didelis žanrų persidengimas, svarbu embedding modeliui



Pav. 3. Top 20 žaidimų žanrų / tagų pagal žaidimų skaičių

3.3 Koreliacijos matrica

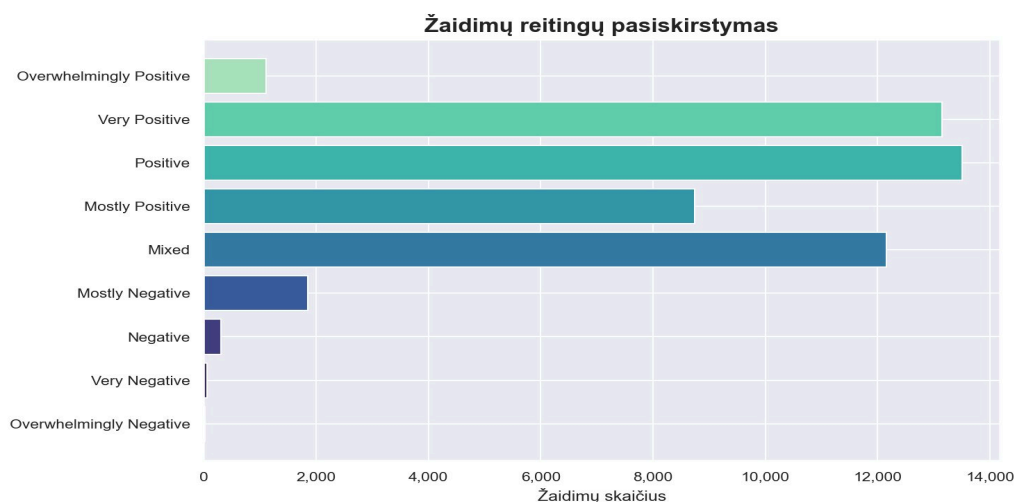
- Kaina ↔ Avg valandos: $r=0.25$ – brangūs žaidimai žaidžiami šiek tiek ilgiau
- Kaina ↔ Metacritic: $r=0.24$ – AAA žaidimai brangiau ir geriau įvertinti kritikų
- Apžvalgos ↔ Avg valandos: $r=0.22$ – populiarūs žaidimai žaidžiami ilgiau
- Visos kitos koreliacijos silpnos ($r < 0.1$) – kintamieji daugiausia nepriklausomi



Pav. 4. Koreliacijos matrica: kaina, reitingas, apžvalgos, Metacritic, valandos

3.4 Reitingų pasiskirstymas

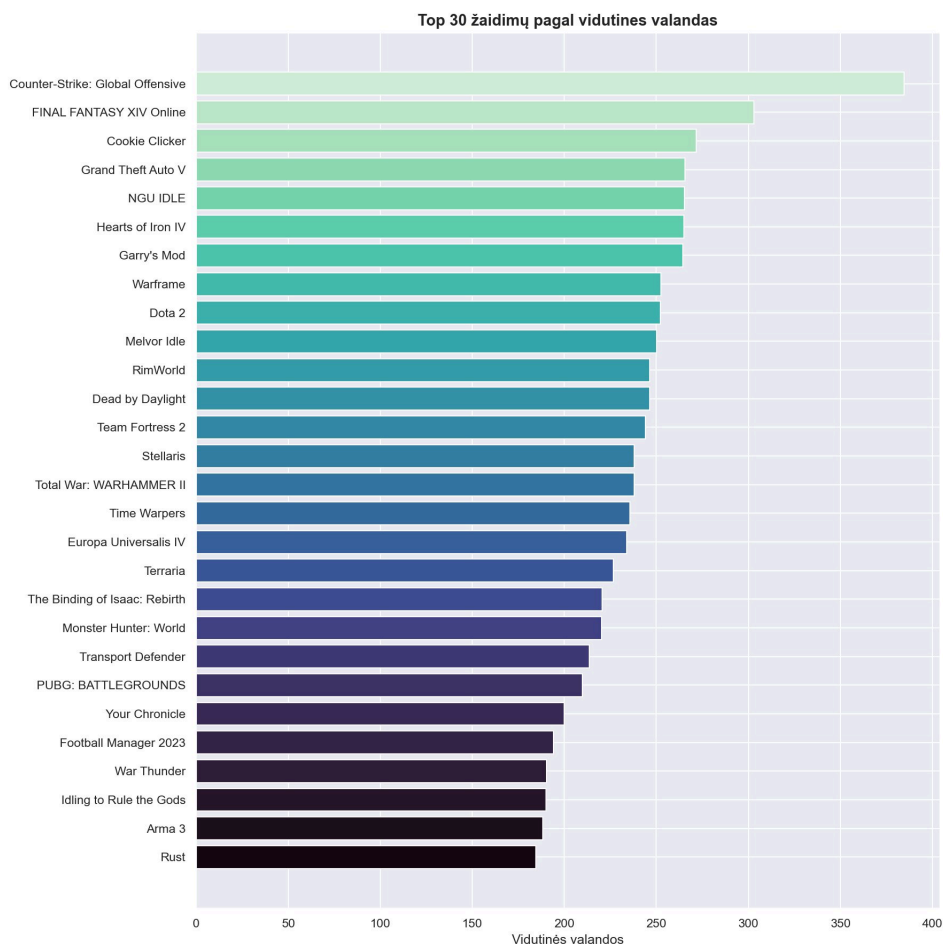
- 'Positive' ir 'Very Positive' – dominuojančios kategorijos (po ~13 000–13 500 žaidimų)
- 'Mixed' – ~12 000 žaidimų, trečia pagal dydį kategorija
- Neigiami reitingai ('Mostly Negative', 'Negative') – mažuma (<2 000)
- 'Overwhelmingly Negative' – praktiškai nėra (duomenų rinkinyje filtruoti)



Pav. 5. Steam žaidimų reitingų pasiskirstymas pagal kategorijas

3.5 Vartotojų elgsena – vidutinės valandos

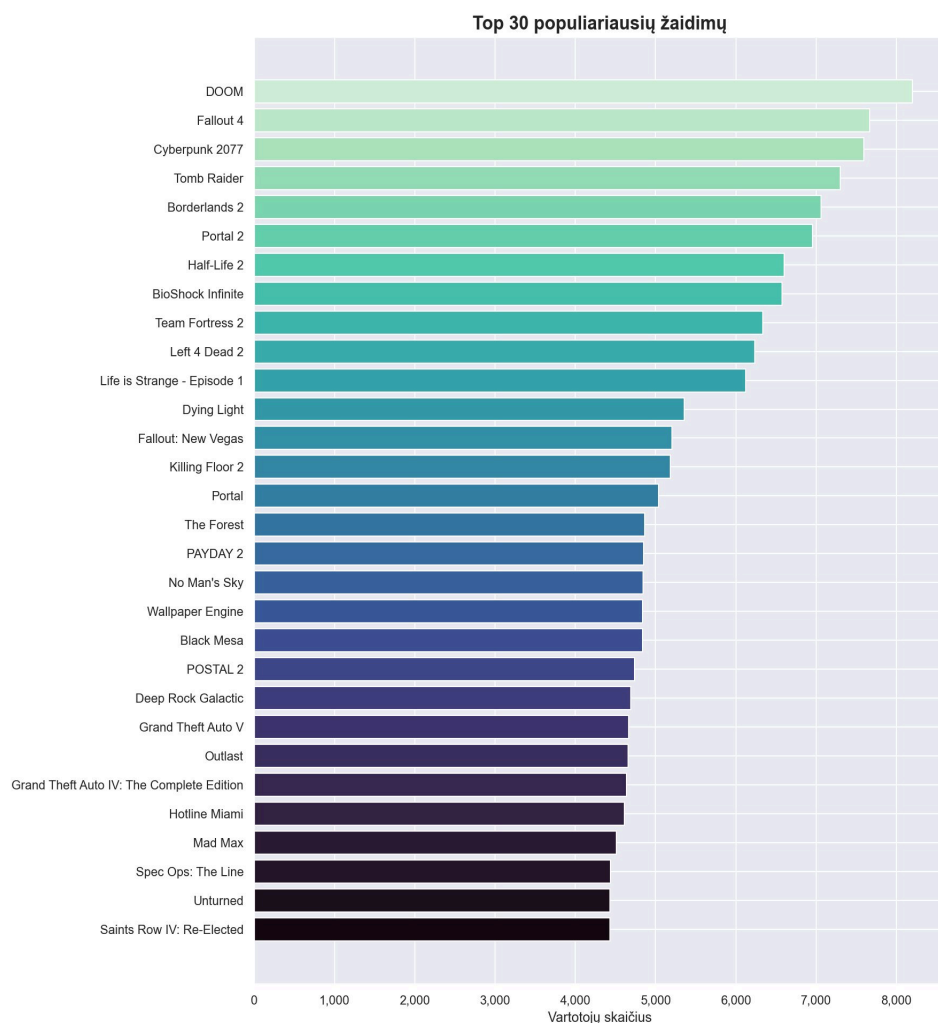
- Counter-Strike: GO – lyderis pagal vidutinės valandas (~380h), tipinis competitive shooter
- FINAL FANTASY XIV Online (~300h) ir Cookie Clicker (~270h) – MMO ir idle žanrai
- Hearts of Iron IV, Europa Universalis IV – strategijos su >230h vidurkiu
- Filtruota: pašalinti Software/Utilities tagų įrašai, žaidimai su <50 vartotojų



Pav. 6. Top 30 žaidimų pagal vidutines žaistų valandas

3.6 Populiariausi žaidimai pagal vartotojų skaičių

- DOOM – lyderis pagal vartotojų skaičių duomenų rinkinyje (~8 000 apžvalgų)
- Fallout 4, Cyberpunk 2077, Tomb Raider – AAA žaidimai top 5
- Pastaba: tai nėra absoliutus Steam populiarumas – tai duomenų rinkinio apžvalgų skaičius
- Portal 2, Half-Life 2 – klasikos su stipriu vartotojų baze

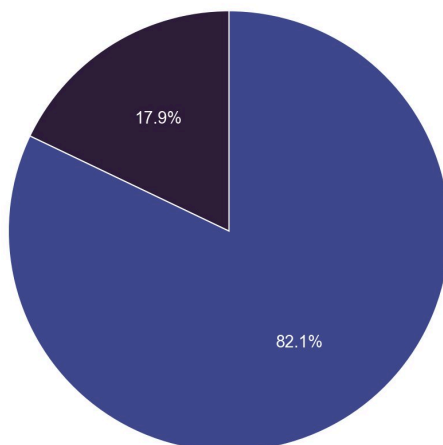


Pav. 7. Top 30 populiariausių žaidimų pagal apžvalgų skaičių duomenų rinkinyje

3.7 Nemokamų vs mokamų žaidimų santykis

- 82.1% žaidimų – mokami ($\text{price_final} > 0$)
- 17.9% žaidimų – nemokama (Free-to-Play)
- F2P žaidimai dažniausiai turi daugiausiai vartotojų (Dota 2, Team Fortress 2, Warframe)

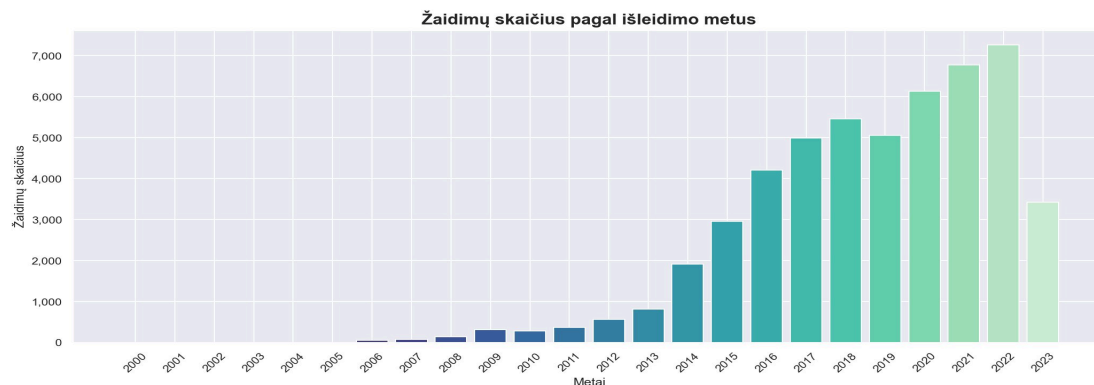
Nemokamų vs mokamų žaidimų santykis



Pav. 8. Nemokamų (17.9%) vs mokamų (82.1%) žaidimų santykis

3.8 Žaidimų skaičius pagal išleidimo metus

- Iki 2012 m. – lėtas augimas (Steam platforma dar tik plėtėsi)
- 2013–2018 m. – spartus augimas, Steam Greenlight/Direct programos atidarymas
- 2022 m. – rekordinis: ~7 200 žaidimų per vienerius metus
- 2023 m. – mažiau (~3 400): duomenų rinkinys apima tik dalį metų



Pav. 9. Žaidimų skaičius pagal išleidimo metus (2000–2023)

4. Semantinė paieška (3 etapas)

4.1 Embeddings generavimas

Naudotas modelis: all-mpnet-base-v2 (Hugging Face sentence-transformers biblioteka). Modelis generuoja 768 dimensijų tankius vektorius. Pasirinktas dėl aukštų MTEB benchmark rodiklių semantinės panašumo paieškos užduotyse.

Teksto formavimo strategija (build_text funkcija):

```
title + genres + tags + tags (x2) + about[:300]
```

Tags kartojami du kartus – boost žanro žodžiams lyginant su aprašymu

Iš viso embed'uota: ~50 872 žaidimų. Embeddings išsaugoti kaip embeddings.npy (float32) ir app_ids.npy. Embedding generavimas atliktas Google Colab aplinkoje (T4 GPU) dėl didelės apimties.

4.2 FAISS vektorinis indeksas

- Naudotas IndexFlatL2 – tikslus Euklido atstumo indeksas
- Prieš indeksavimą visi vektoriai normalizuoti (faiss.normalize_L2)
- Po normalizacijos L2 atstumas ekvivalentus kosinusiniam panašumui
- Indeksas išsaugotas kaip games.index failas – nereikia perkurti kiekvieną kartą

4.3 Paieškos funkcija ir rikiavimo režimai

Įdiegti 4 rikiavimo režimai (Flask UI leidžia pasirinkti):

Režimas	Formulė	Paskirtis
semantic	100% semantic_score	Grynai semantinis – be jokio populiarumo
balanced (default)	80% semantic + 20% positive_ratio	Balansas tarp prasmės ir kokybės
popular	60% semantic + 40% positive_ratio	Labiau pasverta į populiarius žaidimus
logarithmic	80% semantic + 10% log(reviews) + 10% ratio	Mažina outlier efektą didžiuliuose kataloguose

4.4 TF-IDF vs Semantinė paieška (palyginimas)

Atliktas kiekybinis palyginimas tarp klasikinės TF-IDF paieškos ir semantinės embeddings paieškos:

- TF-IDF: greita, tačiau nesupranta sinonimų ir konteksto (pvz. 'shooter' neranda 'FPS')
- Semantinė: lėtesnė inicializacija, tačiau žymiai prasmingesniai rezultatai
- Overlap@10 (bendrų rezultatų): vidutiniškai 2–4 iš 10 – metodai papildo vienas kitą
- Pavyzdys: užklausa 'relaxing city builder no combat' – semantinė grąžina Stardew Valley, TF-IDF – Cities: Skylines

4.5 Kešavimas

Paieškos rezultatai kešuojami PostgreSQL `search_cache` lentelėje. Raktas – MD5 hash iš (query + top_k + mode). `ON CONFLICT DO NOTHING` garantuoja saugumą be dublikatų.

4.6 Steam Web API integracija

Žaidimų viršelio paveikslėliams ir kainoms gauti naudojamas oficialus Steam Web API (store.steampowered.com/api/appdetails). API nemokamas ir nereikalauja autentifikacijos.

Parametras	Reikšmė
Endpoint	<code>https://store.steampowered.com/api/appdetails</code>
Parametrai	<code>appids={app_id}&filters=basic,price_overview</code>
Grąžina	<code>header_image</code> URL, <code>price_overview.final</code> (centais, pvz. 1499 = \$14.99)
Rate limiting	<code>time.sleep(0.3)</code> tarp kvietimų – max ~3 užklausos per sekundę
Cache	Jei <code>header_image</code> jau yra DB – API nekviečiamas (DB cache pirmenybė)
Timeout	5 sekundžių – jei Steam neatsiliepia, grąžinama None
max_calls	Vieno batch'o limitas: 50 užklausų – apsauga nuo per didelio kiekio

API kvietimai atliekami dviem atvejais: pradinis žaidimų papildymas (`enrich_batch`) ir atskiri kvietimai kai vartotojas atidaro žaidimo puslapį. Duomenys atnaujinami DB ir kešuojami – pakartotinis peržiūrėjimas nekviečia API.

5. Rekomendacijų sistema (4 etapas)

5.1 User-Item matrica

Vartotojas $\times$ žaidimas matrica sukurta iš recommendations lentelės. Svoris – $\log(1+\text{hours})$ su koregavimu neigiamoms apžvalgoms:

- $\log(1+\text{hours})$ – sumažina didelio žaidimo laiko dominavimą ($\log(1001) \approx 6.9$, o ne 1001)
- $\text{is_recommended}=\text{False} \rightarrow \text{rating} \times 0.1$ – informacija išlaikoma, bet su minimaliu svoriu
- Matrica konvertuota į `scipy.sparse.csr_matrix` – efektyvus atminties naudojimas
- Tankis (density): ~ 0.001 – labai reta matrica, tipiškas rekomendacijų sistemoms

5.2 SVD modelis

TruncatedSVD iš scikit-learn bibliotekos. Matrica skaidoma: $M \approx U \times \Sigma \times V^T$

- `n_components=50` latentinių dimensijų
- Paaiškinta dispersija: $\sim 35\%$ su 50 komponentais
- Žanro boost: jei žaidimų žanrai sutampa – panašumo balas $\times 1.2$ (20% padidinimas)
- SVD faktoriai išsaugoti: `svd_user_factors.npy`, `svd_item_factors.npy`, `game_ids.npy`

5.3 NCF modelis (Neural Collaborative Filtering)

PyTorch pagrindu sukurtas neuroninis modelis. Architektūra:

Sluoksnis	Dydis	Aprašymas
user_emb	$n_users \times 64$	Vartotojo embedding sluoksnis
item_emb	$n_items \times 64$	Žaidimo embedding sluoksnis
concat	128	Sujungiama [user_emb, item_emb]
MLP 1	$128 \rightarrow 128$, ReLU + Dropout(0.2)	Pirmasis pilnai sujungtas sluoksnis
MLP 2	$128 \rightarrow 64$, ReLU	Antrasis sluoksnis
Output	$64 \rightarrow 1$, Sigmoid	Tikimybė $P(\text{vartotojas žais}) \in (0,1)$

Treniravimo detalės:

- Negative sampling: balansuoti teigiami ir neigiami pavyzdžiai (1:1)
- Loss funkcija: BCELoss (Binary Cross-Entropy)
- Optimizatorius: Adam, $\text{lr}=0.001$
- Batch size: 4096, Epochs: 10 (geriausias variantas)

5.4 NCF eksperimentai – 32 versijos

Sistemiškai išbandytos 32 NCF konfigūracijos dviejose grupėse. Kiekviena versija keičia vieną hiperparametrą ar architektūros elementą, kad būtų galima izoliuoti poveikį.

Grupė A – Be papildomų features (v1–v15, tik user_id + app_id):

Ver.	EMB	EP.	BATCH	LR	Pakeitimas	Acc.	Prec.	F1
v1	64	20	2048	0.001	Bazinis	0.8432	0.8608	0.9103
v2	32	20	2048	0.001	EMBED 64→32	0.8451	0.8649	0.9110
v3	128	20	2048	0.001	EMBED 64→128	0.8411	0.8605	0.9090
v4	32	50	2048	0.001	EPOCHS 20→50 (overfitting)	0.8380	0.8590	0.9072
v5	32	10	2048	0.001	EPOCHS 20→10	0.8473	0.8646	0.9125
v6	32	10	512	0.001	BATCH 2048→512	0.8464	0.8646	0.9119
v7	32	10	4096	0.001	BATCH 2048→4096 ★ best A	0.8477	0.8679	0.9123
v8	32	10	4096	0.01	LR 0.001→0.01	0.8406	0.8622	0.9085
v9	32	10	4096	0.0001	LR 0.001→0.0001 (Recall≈1)	0.8245	0.8248	0.9037
v10	32	10	4096	0.001	Dropout 0.2→0.5	0.8471	0.8712	0.9115
v11	32	10	4096	0.001	ReLU→LeakyReLU	0.8475	0.8661	0.9124
v12	32	10	4096	0.001	Adam→SGD (Recall=1.0, nesimoko)	0.8239	0.8239	0.9035
v13	32	10	4096	0.001	Adam→RMSprop	0.8472	0.8671	0.9121
v14	32	10	4096	0.001	MLP 2→3 sluoksniai	0.8473	0.8695	0.9119
v15	32	10	4096	0.001	MLP 2→1 sluoksnis	0.8461	0.8713	0.9108

Grupė B – Su papildomais features (v16–v32, + price, ratio, genres):

Ver.	EMB	EP.	BATCH	LR	Pakeitimas	Acc.	Prec.	F1
v16	32	10	4096	0.001	Bazinis su features (price, ratio, genres)	0.8472	0.8677	0.9120
v17	32	10	4096	0.001	weight_decay=1e-4	0.8476	0.8649	0.9126
v18	32	10	4096	0.001	LR Scheduler StepLR(step=3, gamma=0.5)	0.8470	0.8625	0.9126
v19	32	10	4096	0.001	Tanh aktyvacija	0.8472	0.8661	0.9122
v20	32	17*	4096	0.001	Early Stopping (patience=3)	0.8474	0.8657	0.9124
v21	32	10	4096	0.001	NEG_RATIO=2 (1:2)	0.8477	0.8641	0.9128
v22	32	10	4096	0.001	USER_MIN=100 (mažiau duomenų)	0.8440	0.8560	0.9115
v23	32	10	4096	0.001	GAME_MIN=500	0.8558	0.8710	0.9187
v24	64	15	4096	0.001	EMBED=64 + EPOCHS=15	0.8569	0.8747	0.9189
v25	64	15	4096	0.001	BatchNorm (overfitting)	0.8484	0.8795	0.9129
v26	64	15	4096	0.001	weight_decay=1e-3	0.8554	0.8737	0.9181
v27	64	15	4096	0.001	weight_decay=1e-5	0.8393	0.8742	0.9078

v28 ★	64	15	4096	0.001	GAME_MIN=1000 — GERIAUSIAS	0.8605	0.8704	0.9224
v29	64	15	4096	0.001	NEG_RATIO=3	0.8602	0.8727	0.9220
v30	64	15	4096	0.001	NCF + MultiheadAttention (4 galvos)	0.8552	0.8794	0.9181
v31	64	20	4096	0.001	Transformer + Features Sequence + CLS token	0.8597	0.8759	0.9213
v32	64	30	4096	0.0005	Transformer EPOCHS=30, LR=0.0005	0.8596	0.8759	0.9212

* v20 sustojo 17 epochoje dėl Early Stopping (patience=3)

Svarbiausios išvados iš eksperimentų:

- Geriausias modelis – v28 (F1=0.9224): GAME_MIN=1000 suteikė didžiausią pagerinimą – duomenų kokybė svarbesnė nei architektūra
- GAME_MIN svarba: v1→v23→v28 trajektorija (F1: 0.9103→0.9187→0.9224) – filtravimas labiausiai padėjo
- Transformer (v30–v32) neaplenkia paprasto NCF v28 – sudėtingesnė architektūra nepadeda esant ribotiems duomenims
- SGD (v12) ir LR=0.0001 (v9): Recall≈1.0 – degenerate sprendimas, modelis viską klasifikuoja kaip teigiamą
- BatchNorm (v25): Precision gerėja, bet F1 blogėja – overfitting požymis
- weight_decay=1e-4 optimalus: per stipri (1e-3) ir per silpna (1e-5) regularizacija blogina rezultatus
- USER_MIN=100 (v22): mažiau duomenų = blogesnis modelis – filtravimas per griežtas

5.5 NCF v28 – treniravimo detalės (Colab)

Galutinis v28 modelis treniruotas Google Colab aplinkoje (T4 GPU) naudojant Python skriptą su detaliais komentarais. Žemiau pateikiamos visos techninės treniravimo detalės:

Parametras	Reikšmė ir paaiškinimas
EMBED_DIM	64 – kiekvienas vartotojas ir žaidimas aprašomas 64 skaičių vektoriumi
EPOCHS	15 – modelis 15 kartų peržiūrėjo visus treniravimo duomenis
BATCH_SIZE	4096 – kiek pavyzdžių paduodama vienu metu prieš atnaujinant svorius
LR	0.001 – Adam optimizatoriaus mokymosi greitis
WEIGHT_DECAY	1e-4 – L2 regularizacija, baudžia per didelius svorius (prevencija overfitting)
NEG_RATIO	2 – vienam teigiamam pavyzdžiui du neigiami (1:2 balansas treniravimui)
USER_MIN	50 – vartotojai su mažiau nei 50 apžvalgų pašalinami iš duomenų
GAME_MIN	1000 – ★ pagrindinis pagerinimas: tik žaidimai su ≥1000 apžvalgų

TEST_RATIO	0.20 – 80% treniravimui, 20% testavimui (shuffle su RANDOM_SEED=42)
Device	CUDA (T4 GPU) – ~10x greitesnis nei CPU

Train / Test strategija

Svarbi detalė: treniravimo ir testavimo duomenys balansuojami skirtingai:

- Train (balance=True): balansuojama 1:2 – vienam teigiamam du neigiami. Būtina nes natūraliai ~83% įrašų yra 'rekomenduoja' – nebalansuoti duomenys verčia modelį viską spėti kaip teigiama
- Test (balance=False): natūralus pasiskirstymas – atspindi realų pasaulį. Balansavimas teste iškraipytų metrikas ir nerodytų tikrų sąlygų
- Split: duomenys sumaišomi (shuffle) su RANDOM_SEED=42 → pirmieji 80% = train, paskutiniai 20% = test

Batch inference (OOM prevencija)

evaluate() funkcija naudoja batch inference – spėjimai skaičiuojami ne visi vienu metu, o dalimis (batch_size=4096). Tai išsprendžia GPU atminties perpildymo (Out Of Memory) problemą su dideliais test rinkiniais. squeeze(dim=-1) naudojamas vietoj squeeze() – apsauga nuo klaidos kai batch_size=1.

Modelio išsaugojimas

Po treniravimo išsaugomi du failai į Google Drive:

- ncf_v28_final.pth – modelio svoriai PyTorch formatu (torch.save(model.state_dict()))
- ncf_v28_final_metadata.pkl – metaduomenys pickle formatu: user_to_idx, item_to_idx, feature_cols, n_users, n_items, n_features, embed_dim

Flask aplikacija inference metu įkelia abu failus – metadata reikalinga konvertuoti vartotojo/žaidimo ID į modelio indeksus, o feature_cols sąrašas užtikrina teisingą features eilės tvarką.

6. RAG + Gemini (5 etapas)

6.1 RAG pipeline aprašymas

RAG (Retrieval-Augmented Generation) – technika, kuri sujungia informacijos paiešką su kalbos modelio generavimu. Tai leidžia modeliui kalbėti apie konkrečius Steam žaidimus remdamasis faktiniais duomenimis iš DB.

Pipeline veikimas:

- Retrieve: FAISS paieška → top-5 semantiškai artimų žaidimų
- Rerank: semantic 70% + popularity 30% – filtruojamos šiukšlės
- Augment: žaidimų duomenys (title, genres, rating, price, about) įterpiami į prompt
- Generate: Gemini 2.5 Flash generuoja lietuvišką paaiškinimą

6.2 Gemini API integracija

Pradžioje planuotas Azure OpenAI (GPT-4o), tačiau API quota buvo blokuota. Sistema perkelta į Google Gemini (Google AI Studio). API nemokamas iki tam tikros kvietimų ribos.

Parametras	Reikšmė
Modelis	gemini-2.5-flash – greitas ir ekonomiškas, tinka RAG paaiškinimams
Biblioteka	google-genai (naujasis Client() API, ne senasis google.generativeai)
Autentifikacija	GEMINI_API_KEY iš .env failo – gautas iš aistudio.google.com
Endpoint	client.models.generate_content(model, contents=prompt)
Flask integracija	/api/explain?q={query} – async endpoint, nablokuoja puslapio krovimo
Kada kviečiamas	Kiekvienai paieškos užklausai – vienas Gemini iškvietimas fone
Atsakymo laikas	Vidutiniškai 1–2 sekundės – todėl async (puslapis nekelia)
Kalbos palaikymas	Lietuvių (LT) ir anglų (EN) – nustatoma pagal language parametą

Saugumo pastaba: treniravimo metu du Gemini API raktai buvo atsitiktinai eksponuoti GitHub'e ir turėjo būti nedelsiant ištrinti iš aistudio.google.com. Nuo to laiko API raktai saugomi tik .env faile kuris pridėtas į .gitignore.

6.3 Prompt engineering

Optimizuotas prompt šablonas su aiškia rolė, struktūrizuotu kontekstu ir ilgio apribojimu (max 150 žodžių). Palaikoma lietuvių ir anglų kalba pagal language parametą.

Prompt struktūra: rolė → rasti žaidimai (sunumeruoti su faktais) → 3 užduotys → kalbos instrukcija

7. SHAP ir personalizacija (6 etapas)

7.1 SHAP integracija

SHAP (SHapley Additive exPlanations) – metodas paaiškinti modelio sprendimus. Kiekvienam žaidimui apskaičiuojamos feature svarbumo reikšmės, parodomos vartotojui kaip horizontalus bar chart.

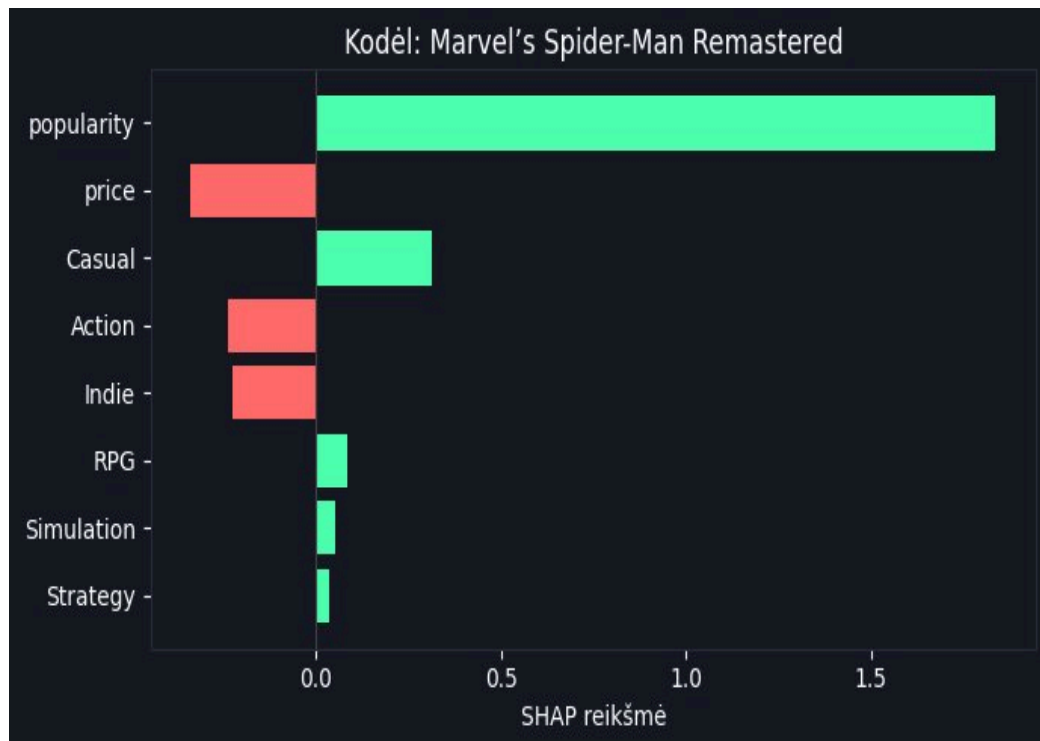
Implementacija:

- Modelis: GradientBoostingClassifier – pasirinktas dėl tikslaus TreeExplainer veikimo su medžių modeliais
- Features (17 viso): 15 žanrų binary flags + price_norm + popularity
- Tikslas (y): ar žaidimas turi positive_ratio $\geq 80\%$
- Teigiama SHAP reikšmė $\rightarrow$ feature didina rekomendacijos tikimybę
- Neigiama SHAP reikšmė $\rightarrow$ feature mažina tikimybę

7.2 SHAP vizualizacija

Kiekvienam žaidimui generuojamas PNG failas su tamsiu fonu (atitinka Flask UI stilių). Failai pavadinti pagal app_id (pvz. shap_1817070.png) ir patiekiami per Flask /static/shap/ URL.

Žemiau – SHAP vizualizacijos pavyzdys žaidimui Marvel's Spider-Man Remastered:



Pav. 1. SHAP reikšmės – Marvel's Spider-Man Remastered. Popularity dominuoja (didelė teigiama įtaka), o kaina – neigiama (žaidimas brangus). Action ir Indie žanrai veikia priešingai.

7.3 User Profile Vector

Vartotojo paieškų istorija transformuojama į personalizuotos paieškos vektorių:

- Kiekvieno peržiūrėto žaidimo embedding ištraukiamas iš embeddings.npy
- Eksponentinis svoris: senesni žaidimai gauna mažesnį svorį (nauji interesai svarbiau)
- Profilio vektorius = eksponentinis vidurkis → FAISS paieška
- Jau matyti žaidimai pašalinami iš rezultatų (similarity = -1)

7.4 Paieškos istorija

Kiekviena paieška išsaugoma search_history DB lentelėje (session_id, query, results JSON, timestamp). Top 5 rezultatai saugomi JSON formatu. Istorija naudojama build_user_profile() funkcijai personalizuotų rekomendacijų generavimui.

8. Flask Web Aplikacija (7 etapas)

8.1 Architektūra ir dizainas

Backend: Python Flask su SQLAlchemy 2.0. Visa business logika atskirta į modulius (search, recommend, shap, rag). Async Gemini užklausos vykdomos per /api/explain endpoint atskirame thread'e – puslapis atidaromas iš karto, AI tekstas pasirodo po 1–2 sekundžių be perkrovimo.

Frontend: vanilla HTML/CSS/JS be papildomų frameworks. Dizainas tamsus (dark theme), Google Fonts: Bebas Neue antraštėms, DM Sans tekstui. CSS custom properties vieningai valdomi per :root. Nav su backdrop-filter: blur(12px) – glass efektas.

8.2 Puslapiai ir funkcionalumas

Puslapis	URL	Funkcionalumas
index.html	/ ir /search	Paieška, 7 stulpelių rezultatų palyginimas, SVD rekomendacijos, Gemini paaiškinimas async, paieškos istorijos chips
game.html	/game/<app_id>	Hero su blur fono efektu, žaidimo statistika, SVD + NCF v28 rekomendacijos, SHAP grafikas + badge'ai
compare.html	/compare	SVD vs NCF metrikų palyginimas – Chart.js interaktyvūs grafikai Precision@K ir Recall@K
profile.html	/profile	Žanrų statistika su progress barais, visos paieškos su miniatiūromis, istorijos valymas

8.3 index.html – 7 paieškos stulpeliai

CSS grid su 7 stulpeliais (grid-template-columns: repeat(7, 1fr)). Kiekvienas stulpelis – atskiras paieškos algoritmas, leidžiantis tiesioginį vizualinį palyginimą:

Stulpelis	Ikona	Formulė / principas	Spalva UI
Keyword		SQL ILIKE – klasikinė raktažodžių paieška	#ff6b6b (raudona)
KNN		TF-IDF cosine similarity	#ff9f43 (oranžinė)
Subalansuotas		$sem \times 0.8 + positive_ratio \times 0.2$	#4fffb0 (žalia)
Semantinis		$sem \times 1.0$ – grynas embedding panašumas	#c864ff (violetinė)
Populiarus		$sem \times 0.6 + positive_ratio \times 0.4$	#00b4ff (mėlyna)
Logaritminis		$sem \times 0.8 + \log(reviews) \times 0.1 + ratio \times 0.1$	#ffc107 (geltona)
Custom		3 range slideriai (semantika, reitingas %, apžvalgos). Suma turi = 100%. Mygtukas TAIKYTI perkrauna paiešką su naujais svoriais.	#ff9f43 (oranžinė)

Eilutės kiekviename stulpelyje: eilės numeris (Bebas Neue), 56×32px miniatiūra, pavadinimas (ellipsis), spalvotas reitingo badge (žalia ≥80%, geltona ≥60%, raudona <60%), balo reikšmė.

8.4 game.html – žaidimo puslapis

Hero sekcija: header_image kaip fonas su filter: blur(20px) brightness(0.3) + transform: scale(1.1). Virš fono – žaidimo viršelis (232×130px), pavadinimas, žanrų tagų eilutė, stat blokai.

- SVD sekcija – Collaborative Filtering badge, 5 kortelių grid (accent spalva: #00b4ff)
- NCF v28 sekcija – F1: 0.9224 badge, atskiras spalvinis identitetas (accent3: #a78bfa violetinė)
- SHAP sekcija – PNG iš /static/shap/shap_{app_id}.png + feature badge'ai: žalia teigiamoms, raudona neigiamoms SHAP reikšmėms

8.5 profile.html – vartotojo profilis

- Session identifikavimas per Flask session_id (automatiškai generuojamas)
- Žanrų statistika: skaičiuojama iš paieškos rezultatų genres – 4-stulpelinis grid su animuotais progress barais
- Paieškos istorija: kiekviena užklausa su timestamp ir žaidimų miniatiūromis (24×14px)
- POST /api/clear_history – išvalo DB search_history pagal session_id

8.6 GitHub repozitorija

- URL: github.com/tkeatas92/baigiamasis_final
- Branch strategija: dev → main
- Struktūra: /app (Flask + templates + static), /src (ML moduliai), /notebooks, /data, /models
- README.md: projekto aprašymas, tech stack, paleidimo instrukcijos

9. Rezultatai ir išvados

9.1 Modelių palyginimo rezultatai

Žemiau pateikiama pilna modelių palyginimo lentelė. NCF v28 buvo treniruotas Google Colab T4 GPU aplinkoje su 80/20 train/test split strategija ir natūraliu testavimo pasiskirstymu.

Modelis	Accuracy	Precision	Recall	F1	Pastaba
SVD (scikit-learn)	-	~0.65	~0.72	-	Precision@10, Recall@10 metrika
NCF v1 (baseline)	0.8432	0.8608	0.9659	0.9103	Embed=64, Ep=20, Batch=2048
NCF v7 (geriausias A)	0.8477	0.8679	0.9615	0.9123	Batch=4096, Ep=10 – be features
NCF v17 (geriausias B)	0.8476	0.8649	0.9659	0.9126	+ weight_decay=1e-4
NCF v23 (GAME_MIN=500)	0.8558	0.8710	0.9718	0.9187	Pirmasis duomenų filtravimo šuolis
NCF v24 (EMBED=64+EP=15)	0.8569	0.8747	0.9677	0.9189	Didesnis embedding + daugiau epochų
NCF v28 ★ (GAME_MIN=1000)	0.8605	0.8704	0.9809	0.9224	GERIAUSIAS – duomenų kokybė
NCF v31 (Transformer)	0.8597	0.8759	0.9716	0.9213	Sudėtingesnė architektūra – prastesnė

9.2 Paieškos metodų palyginimas

Metodas	Technologija	Privalumai	Trūkumai
Keyword	SQL ILIKE	Greitas, tikslus žodžiams	Nesupranta prasmės, sinonimų
KNN	TF-IDF cosine	Geresnis nei Keyword	Vis dar žodžių paieška, ne semantika
Semantinis	FAISS + mpnet-base-v2	Supranta prasmę, sinonimus, kontekstą	Lėtesnė inicializacija
Subalansuotas	sem×0.8 + ratio×0.2	Balansas tarp prasmės ir kokybės	Gali sumažinti nišinius rezultatus
Logaritminis	sem×0.8 + log×0.1 + ratio×0.1	Mažina outlierių įtaką	Sudėtingesnė formulė
Custom	Vartotojo svoriai	Maksimalus lankstumas	Reikia vartotojo supratimo

9.3 Pagrindinės išvados

- Duomenų kokybė > architektūros sudėtingumas: GAME_MIN=1000 (v28) davė didesnį F1 pagerinimą nei bet koks architektūrinis pakeitimas (Transformer, MultiheadAttention ir kt.)
- GAME_MIN trajektorija: v1→v23→v28 (F1: 0.9103→0.9187→0.9224) – kiekvienas filtravimo griežtinimas davė pastebimą pagerėjimą
- Train/Test split strategija svarbi: train balansuojama (1:2) nes natūraliai 83% teigiamų – be balansavimo modelis spėtų viską kaip teigiama. Test naudoja natūralų pasiskirstymą
- Semantinė paieška (all-mpnet-base-v2 + FAISS) teikia prasmingesnius rezultatus nei TF-IDF ir Keyword – ypač abstrakčioms užklausoms ('relaxing game no combat')
- NCF (F1=0.9224) gerokai lenkia SVD – neuroniniai embedding modeliai geriau išmoka kompleksines vartotojų-žaidimų priklausomybes nei matricos faktorizavimas
- RAG + Gemini 2.5 Flash teikia kontekstualius lietuviškus paaiškinimus – perėjimas nuo Azure OpenAI (blokuota quota) prie Gemini buvo sėkmingas
- SHAP vizualizacija sėkmingai paaiškina kiekvieną rekomendaciją: popularity, kaina, žanrai su kiekybiniais SHAP įverčiais
- Steam Web API integracija praturtina rezultatus viršelio paveikslėliais ir tikslesnėmis kainomis su DB laikinu saugojimu (cache).

9.4 Realizuotos sistemos komponentai

✓	Komponentas	Pastaba
✓	PostgreSQL DB + 6 lentelės (games, users, recommendations, search_cache, model_results, search_history)	SQLAlchemy 2.0, psycopg2 batch insert
✓	EDA – 8 grafikai (kainos, žanrai, koreliacijos, reitingai, elgsena, populiarumas, F2P, metai)	Seaborn/Matplotlib, darkgrid tema
✓	Embeddings generavimas – all-mpnet-base-v2, 768d, ~50 872 žaidimų	Google Colab T4 GPU, ~13 min
✓	FAISS IndexFlatL2 semantinė paieška – 7 režimai (Keyword, KNN, balanced, semantic, popular, log, custom)	MD5 hash cache DB
✓	TF-IDF vs Semantic palyginimas	CSV rezultatai, Overlap@10 metrika
✓	SVD rekomendacijos (TruncatedSVD, n=50) + žanro boost 20%	svd_user_factors.npy, svd_item_factors.npy
✓	NCF modelis (PyTorch) – 32 versijų eksperimentai, geriausias v28 F1=0.9224	Google Colab T4, ncf_v28_final.pth
✓	Steam Web API integracija – viršelio paveikslėliai, kainos	Rate limit: 0.3s, DB cache
✓	RAG + Gemini 2.5 Flash – lietuviški/angliški paaiškinimai	Async /api/explain, google-genai
✓	SHAP + GradientBoosting – 17 features, TreeExplainer	PNG per /static/shap/
✓	User Profile Vector – eksponentinis svoris, personalizuota paieška	search_history DB lentelė
✓	Flask UI – 4 puslapiai (index, game, compare, profile), 7 paieškos stulpeliai	Dark theme, Bebas Neue + DM Sans

✓	GitHub repozitorija	github.com/tkestas92/baigiamasis_final, dev→main
---	---------------------	---

9.5 Galimos tobulinimo kryptys

- PUBG / battle royale žaidimų FAISS retrieval problema – žanro vektoriai per panašūs, sunku atskirti nuo kitų action žaidimų
- Realų vartotojų A/B testavimas – šiuo metu metrikos skaičiuotos ant istorinių duomenų, ne realų vartotojų atsiliepimų
- Didesnių duomenų rinkinio naudojimas – su daugiau nei 50k žaidimų ir 7M apžvalgų Transformer architektūra gali pranokti NCF
- Hibridinis modelis: semantinė paieška + NCF kolaboratyvus filtravimas sujungti į vieną galutinį rankerį
- Daugiakalbė paieška – šiuo metu sistema veikia anglų kalba, lietuviška paieška reikautų daugiakalbio embedding modelio
- Realio laiko modelio atnaujinimas – šiuo metu modelis treniruotas vieną kartą, naujų žaidimų/vartotojų reikalauja perkartojimo

Projektas sėkmingai įgyvendino visus nustatytus tikslus: 7 paieškos režimai (Keyword, KNN, semantinis, subalansuotas, populiarius, logaritminis, custom), SVD ir NCF rekomendacijos, Steam Web API ir Gemini RAG integracija, SHAP vizualizacija ir pilna Flask aplikacija su GitHub repozitorija. Svarbiausia eksperimentų išvada: duomenų kokybė svarbesnė nei architektūros sudėtingumas.